LAPORAN PRAKTIKUM
Modul 7: CRUD dan Upload File pada Laravel



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Tugas akhir Modul 7 ini saya kerjakan untuk membuat aplikasi pelaporan banjir bernama LaporBanjir. Aplikasi ini dibangun dengan Laravel 12 dan memakai Laravel Breeze untuk autentikasi. Pada latihan modul, CRUD hanya dipakai untuk mengelola satu tabel berkas milik pengguna yang sedang login. Di tugas akhir ini alurnya berbeda karena ada dua pihak yang terlibat. Warga mengirim laporan dari form publik tanpa akun, sedangkan petugas mengelola laporan setelah login.

1. Memahami penerapan CRUD dan resource controller pada Laravel.
2. Memahami unggah foto melalui form multipart, validasi berkas, disk public, dan symbolic link storage.
3. Membuat tabel laporan, model Laporan, dan LaporanController dengan tujuh method resource.
4. Mengatur route agar form warga dapat diakses tanpa login dan halaman pengelolaan terlindungi middleware auth.
5. Membuat view index, create, show, dan edit yang menampilkan thumbnail, pagination, status, serta pesan berhasil dan pesan galat.
6. Menguji validasi foto jpg, jpeg, dan png dengan ukuran maksimal 2 MB, penggantian foto, penghapusan foto, dan pembatasan akses.
7. Menjelaskan alasan pemakaian Laravel Breeze pada aplikasi ini.

2. Alat dan Bahan

Praktikum ini saya kerjakan pada laptop pribadi dengan Windows 11 Pro, prosesor Intel Core i7-13650HX, dan RAM 16 GB. Sebagian besar perangkat lunak sudah terpasang sejak praktikum sebelumnya. Versi yang saya catat dari mesin ini adalah sebagai berikut.

2.1 Perangkat Lunak

Perangkat Lunak	**Versi / Keterangan**
Sistem operasi	Windows 11 Pro
Bahasa pemrograman	PHP 8.2.12
Framework	Laravel 12.70.1
Paket autentikasi	Laravel Breeze 2.4.2 (Blade)
Database	MariaDB 10.4.32 (XAMPP)
Klien database	phpMyAdmin 5.2.1
Dependency manager	Composer 2.10.3
Build tool	Node.js 25.6.0, npm 11.8.0, Vite

Perangkat Lunak	Versi / Keterangan
Web server	PHP built-in server (php artisan serve)
Editor	Visual Studio Code 1.143.1
Browser	Google Chrome 156.0.8078.48
Terminal	PowerShell
Version control	Git 2.53.0

3. Langkah-Langkah Praktikum

3.1 Menyiapkan Project dan Database

Saya memakai folder PRK_TA_7 sebagai lokasi project. Folder itu berisi Laravel 12.70.1 yang sudah terpasang. Nama aplikasi saya atur menjadi LaporBanjir melalui APP_NAME di file .env. Saya juga membuat database db_laporbanjir_modul7 di phpMyAdmin. Nama itu saya pilih untuk membedakan database tugas akhir ini dari database modul 6 yang masih ada di server lokal. Konfigurasi koneksi di .env saya isi dengan DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306, DB_DATABASE=db_laporbanjir_modul7, dan user root tanpa password.

Setelah database siap, saya menjalankan php artisan storage:link. Perintah itu membuat folder public/storage yang menunjuk ke storage/app/public. Tautan ini diperlukan agar foto yang disimpan di disk public bisa dibuka lewat URL /storage/...

3.2 Memasang Laravel Breeze

Autentikasi saya serahkan ke Laravel Breeze. Saya menjalankan composer require laravel/breeze --dev, lalu php artisan breeze:install blade. Setelah itu saya menjalankan npm install dan npm run build, kemudian php artisan migrate. Breeze menambahkan halaman login, register, lupa password, profil, layout Blade, dan komponen Tailwind. Saya memilih Breeze karena kebutuhan aplikasi ini hanya autentikasi petugas, bukan pengelolaan peran yang rumit. Dengan Breeze, saya tidak perlu menulis ulang form login dan middleware auth. Halaman warga tetap memakai layout guest, sedangkan halaman petugas memakai layout app yang disediakan Breeze.

3.3 Membuat Model dan Migration Laporan

Saya membuat model dan migration sekaligus dengan php artisan make:model Laporan -m. Migration itu saya isi dengan kolom nama_pelapor, no_hp, lokasi, kecamatan, tinggi_air, keterangan, foto, status, dan timestamps. Kolom keterangan boleh kosong. Kolom foto menyimpan path file, bukan isi file. Status saya beri nilai bawaan Menunggu karena laporan baru belum ditangani petugas.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('laporan', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelapor');                      // Nama warga pelapor
            $table->string('no_hp');                             // Nomor HP pelapor
            $table->string('lokasi');                            // Lokasi banjir
            $table->string('kecamatan');                         // Kecamatan
            $table->integer('tinggi_air');                       // Tinggi air (cm)
            $table->text('keterangan')->nullable();              // Keterangan (boleh kosong)
            $table->string('foto');                              // Path foto bukti di storage
            $table->string('status')->default('Menunggu');       // Status penanganan
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('laporan');
    }
};

```

Model Laporan saya arahkan ke tabel laporan dan saya isi $fillable supaya kolom-kolom tersebut bisa disimpan lewat create() dan update().

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Laporan extends Model
{
    protected $table = 'laporan'; // Nama tabel yang dihubungkan

    // Kolom yang boleh diisi secara massal (mass assignment)
    protected $fillable = [
        'nama_pelapor',
        'no_hp',
        'lokasi',
        'kecamatan',
        'tinggi_air',
        'keterangan',
        'foto',
        'status',
    ];
}

```

3.4 Membuat LaporanController

Controller dibuat dengan php artisan make:controller LaporanController --resource. Ada tujuh method yang saya isi: index, create, store, show, edit, update, dan destroy. Method index mengambil lima laporan terbaru dengan paginate(5). Method store memvalidasi input, menyimpan foto ke disk public, lalu menulis data ke tabel. Method update memeriksa apakah petugas memilih foto baru. Kalau ada, foto lama dihapus dari storage sebelum foto baru disimpan. Method destroy menghapus file fisik lebih dulu, kemudian menghapus baris di database.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;   // Untuk mengelola (menghapus) file di storage

class LaporanController extends Controller
{
    // READ: menampilkan daftar laporan beserta thumbnail foto dan pagination
    public function index()
    {
        $laporan = Laporan::latest()->paginate(5);
        return view('laporan.index', compact('laporan'));
    }

    // CREATE: menampilkan form pelaporan warga
    public function create()
    {
        return view('laporan.create');
    }

    // CREATE: memvalidasi dan menyimpan laporan baru
    public function store(Request $request)
    {
        $request->validate([
            'nama_pelapor' => 'required|string|max:255',
            'no_hp'        => 'required|string|max:20',
            'lokasi'       => 'required|string|max:255',
            'kecamatan'    => 'required|string|max:255',
            'tinggi_air'   => 'required|integer|min:0',
            'keterangan'   => 'nullable|string',
            'foto'         => 'required|image|mimes:jpg,jpeg,png|max:2048', // maks. 2 MB
        ]);

        $foto = $request->file('foto');              // Ambil foto dari form
        $path = $foto->store('laporan', 'public');   // Simpan ke storage/app/public/laporan

        Laporan::create([
            'nama_pelapor' => $request->nama_pelapor,
            'no_hp'        => $request->no_hp,
            'lokasi'       => $request->lokasi,
            'kecamatan'    => $request->kecamatan,
            'tinggi_air'   => $request->tinggi_air,
            'keterangan'   => $request->keterangan,
            'foto'         => $path,                 // Path foto di storage
            'status'       => 'Menunggu',            // Status bawaan
        ]);

        return redirect()->route('laporan.create')->with('success', 'Laporan berhasil dikirim.');
    }

```

```php
    // READ: menampilkan detail satu laporan
    public function show(Laporan $laporan)
    {
        return view('laporan.show', compact('laporan'));
    }

    // UPDATE: menampilkan form edit laporan
    public function edit(Laporan $laporan)
    {
        return view('laporan.edit', compact('laporan'));
    }

    // UPDATE: menyimpan perubahan (foto baru bersifat opsional)
    public function update(Request $request, Laporan $laporan)
    {
        $request->validate([
            'nama_pelapor' => 'required|string|max:255',
            'no_hp'        => 'required|string|max:20',
            'lokasi'       => 'required|string|max:255',
            'kecamatan'    => 'required|string|max:255',
            'tinggi_air'   => 'required|integer|min:0',
            'keterangan'   => 'nullable|string',
            'foto'         => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
            'status'       => 'required|in:Menunggu,Diproses,Selesai',
        ]);

        $data = $request->only(
            'nama_pelapor',
            'no_hp',
            'lokasi',
            'kecamatan',
            'tinggi_air',
            'keterangan',
            'status'
        );

        if ($request->hasFile('foto')) {                        // Jika petugas memilih foto baru
            Storage::disk('public')->delete($laporan->foto);    // Hapus foto lama
            $data['foto'] = $request->file('foto')->store('laporan', 'public');
        }

        $laporan->update($data);

        return redirect()->route('laporan.index')
            ->with('success', 'Laporan berhasil diperbarui.');
    }

```

```php
    // DELETE: menghapus foto di storage dan data di database
    public function destroy(Laporan $laporan)
    {
        Storage::disk('public')->delete($laporan->foto); // Hapus file fisik
        $laporan->delete();                              // Hapus data di tabel laporan

        return redirect()->route('laporan.index')->with('success', 'Laporan berhasil dihapus.');
    }
}

```

Aturan validasi foto saya tulis required|image|mimes:jpg,jpeg,png|max:2048 pada store dan nullable|image|mimes:jpg,jpeg,png|max:2048 pada update. required dipakai di store karena warga wajib melampirkan foto. Di update, foto bersifat opsional supaya petugas bisa mengubah status tanpa mengganti foto. no_hp saya validasi sebagai string, bukan integer, agar angka nol di depan nomor HP tidak hilang. tinggi_air saya validasi sebagai integer minimal nol.

3.5 Mendaftarkan Route dan Middleware

Route saya bagi menjadi dua kelompok. Form warga memakai route resource dengan only(['create','store']) di luar middleware auth. Halaman pengelolaan memakai except(['create','store']) di dalam grup middleware auth. Dengan pembagian ini, warga bisa membuka /laporan/create dan mengirim laporan tanpa akun, tetapi /laporan, /laporan/{id}, dan halaman edit hanya bisa dibuka setelah login. Kalau belum login, middleware auth mengarahkan ke /login.

```php
<?php

use App\Http\Controllers\LaporanController;
use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

// Form pelaporan warga: dapat diakses tanpa login
Route::resource('laporan', LaporanController::class)->only(['create', 'store']);

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    // CRUD Laporan oleh petugas: hanya dapat diakses setelah login
    Route::resource('laporan', LaporanController::class)->except(['create', 'store']);
});

require __DIR__.'/auth.php';

```

Route::resource memetakan tujuh route ke tujuh method controller. Saya memeriksa hasilnya dengan php artisan route:list. Route laporan.index, laporan.show, laporan.edit, laporan.update, dan laporan.destroy berada di grup auth, sedangkan laporan.create dan laporan.store berada di luar grup.

3.6 Menambahkan Menu Laporan pada Navigasi

Menu Laporan saya tambahkan di navigasi Breeze, baik versi desktop maupun versi mobile. Menu itu memakai route('laporan.index') dan aktif ketika route saat ini cocok dengan laporan.*.

```blade
<!-- Navigation Links -->
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
    <x-nav-link :href="route('dashboard')"
                :active="request()->routeIs('dashboard')">
        {{ __('Dashboard') }}
    </x-nav-link>
    <x-nav-link :href="route('laporan.index')"
                :active="request()->routeIs('laporan.*')">
        {{ __('Laporan') }}
    </x-nav-link>
</div>

```

3.7 Membuat Form Pelaporan Warga

Form warga memakai layout guest supaya tampilannya sederhana dan tidak menampilkan menu petugas. Form ini memuat nama pelapor, no HP, lokasi, kecamatan, tinggi air, keterangan, dan foto. Atribut enctype="multipart/form-data" wajib ada agar file ikut terkirim. Saya juga menambahkan @csrf, old() pada setiap input, dan komponen x-input-error di bawah input. Dengan old(), isian warga tidak hilang ketika validasi gagal.

```blade
{{-- enctype wajib ada agar foto ikut terkirim --}}
<form action="{{ route('laporan.store') }}" method="POST" enctype="multipart/form-data">
    @csrf

    <div class="mb-4">
        <x-input-label for="nama_pelapor" value="Nama Pelapor" />
        <x-text-input id="nama_pelapor" name="nama_pelapor" type="text"
                      class="mt-1 block w-full"
                      :value="old('nama_pelapor')" required autofocus />
        <x-input-error :messages="$errors->get('nama_pelapor')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="no_hp" value="No. HP" />
        <x-text-input id="no_hp" name="no_hp" type="text" class="mt-1 block w-full"
                      :value="old('no_hp')" required />
        <x-input-error :messages="$errors->get('no_hp')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="lokasi" value="Lokasi" />
        <x-text-input id="lokasi" name="lokasi" type="text" class="mt-1 block w-full"
                      :value="old('lokasi')" required />
        <x-input-error :messages="$errors->get('lokasi')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="kecamatan" value="Kecamatan" />
        <x-text-input id="kecamatan" name="kecamatan" type="text" class="mt-1 block w-full"
                      :value="old('kecamatan')" required />
        <x-input-error :messages="$errors->get('kecamatan')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="tinggi_air" value="Tinggi Air (cm)" />
        <x-text-input id="tinggi_air" name="tinggi_air" type="number" min="0"
                      class="mt-1 block w-full"
                      :value="old('tinggi_air')" required />
        <x-input-error :messages="$errors->get('tinggi_air')" class="mt-2" />
    </div>

```

```blade
        <div class="mb-4">
            <x-input-label for="keterangan" value="Keterangan (opsional)" />
            <textarea id="keterangan" name="keterangan" rows="3"
            <textarea id="keterangan" name="keterangan" rows="3"
                      class="mt-1 block w-full border-gray-300
                             rounded-md shadow-sm">{{ old('keterangan') }}</textarea>
            <x-input-error :messages="$errors->get('keterangan')" class="mt-2" />
        </div>

        <div class="mb-4">
            <x-input-label for="foto" value="Foto Bukti (jpg, jpeg, png - maks. 2 MB)" />
            <input id="foto" name="foto" type="file" accept=".jpg,.jpeg,.png"
                   class="mt-1 block w-full" required>
            <x-input-error :messages="$errors->get('foto')" class="mt-2" />
        </div>

        <x-primary-button>Kirim Laporan</x-primary-button>
    </form>
</x-guest-layout>

```

Input foto memakai accept=".jpg,.jpeg,.png" dan required. Di bawah input ada pesan galat dari $errors->get('foto'). Setelah laporan berhasil dikirim, controller mengembalikan redirect ke form dengan session success. Pesan itu ditampilkan di bagian atas form.

3.8 Membuat Halaman Daftar Laporan

Halaman daftar laporan hanya untuk petugas. Tabel menampilkan nomor, thumbnail, nama pelapor, lokasi, kecamatan, tinggi air, status, tanggal, dan aksi. Thumbnail memakai gambar asli yang sudah diperkecil dengan kelas h-16 w-16 object-cover. Status diberi warna berbeda: kuning untuk Menunggu, biru untuk Diproses, dan hijau untuk Selesai. Di bawah tabel ada links() untuk pagination.

```blade
<table class="w-full border text-left text-sm">
    <thead class="bg-gray-100">
        <tr>
            <th class="border p-2">No</th>
            <th class="border p-2">Foto</th>
            <th class="border p-2">Nama Pelapor</th>
            <th class="border p-2">Lokasi</th>
            <th class="border p-2">Kecamatan</th>
            <th class="border p-2">Tinggi Air</th>
            <th class="border p-2">Status</th>
            <th class="border p-2">Tanggal</th>
            <th class="border p-2">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($laporan as $item)
            <tr>
                <td class="border p-2">
                    {{ $laporan->firstItem() + $loop->index }}
                </td>
                <td class="border p-2">
                    {{-- Thumbnail foto bukti --}}
                    <img src="{{ asset('storage/' . $item->foto) }}"
                         alt="Foto laporan {{ $item->nama_pelapor }}"
                         class="h-16 w-16 object-cover rounded border">
                </td>
                <td class="border p-2">{{ $item->nama_pelapor }}</td>
                <td class="border p-2">{{ $item->lokasi }}</td>
                <td class="border p-2">{{ $item->kecamatan }}</td>
                <td class="border p-2">{{ $item->tinggi_air }} cm</td>
                <td class="border p-2">
                    @if ($item->status === 'Menunggu')
                        <span class="px-2 py-1 rounded bg-yellow-100
                                     text-yellow-700">
                            {{ $item->status }}
                        </span>
                    @elseif ($item->status === 'Diproses')
                        <span class="px-2 py-1 rounded bg-blue-100
                                     text-blue-700">
                            {{ $item->status }}
                        </span>
                    @else
                        <span class="px-2 py-1 rounded bg-green-100
                                     text-green-700">
                            {{ $item->status }}
                        </span>
                    @endif
                </td>
                <td class="border p-2">
                    {{ $item->created_at->format('d-m-Y H:i') }}
                </td>

```

```
                            <td class="border p-2 whitespace-nowrap">
                                <a href="{{ route('laporan.show', $item) }}"
                                    class="text-sky-600">Lihat</a> |
                                <a href="{{ route('laporan.edit', $item) }}"
                                    class="text-yellow-600">Edit</a> |
                                <form action="{{ route('laporan.destroy', $item) }}"
                                      method="POST" class="inline">
                                    @csrf
                                    @method('DELETE')
                                    <button type="submit" class="text-red-600"
                                        onclick="return confirm('Yakin ingin menghapus ' +
                                            'laporan ini?')">
                                        Hapus
                                    </button>
                                </form>
                            </td>
                        </tr>
                    @empty
                        <tr>
                            <td colspan="9" class="border p-2 text-center">
                                Belum ada laporan yang dikirim.
                            </td>
                        </tr>
                    @endforelse
                </tbody>
            </table>

            <div class="mt-4">{{ $laporan->links() }}</div>

```

Tombol Lihat mengarah ke halaman detail, Edit ke form edit, dan Hapus mengirim form DELETE dengan konfirmasi. Konfirmasi saya buat dengan onclick="return confirm(...)" supaya petugas tidak menghapus laporan secara tidak sengaja.

3.9 Membuat Halaman Detail

Halaman detail menampilkan seluruh data laporan dan foto bukti. Ukuran foto saya batasi dengan max-w-sm agar tidak memenuhi layar. Di bagian bawah ada tautan Edit dan Kembali.

```blade
<p><strong>Nama Pelapor:</strong> {{ $laporan->nama_pelapor }}</p>
<p><strong>No. HP:</strong> {{ $laporan->no_hp }}</p>
<p><strong>Lokasi:</strong> {{ $laporan->lokasi }}</p>
<p><strong>Kecamatan:</strong> {{ $laporan->kecamatan }}</p>
<p><strong>Tinggi Air:</strong> {{ $laporan->tinggi_air }} cm</p>
<p><strong>Keterangan:</strong> {{ $laporan->keterangan ?? '-' }}</p>
<p><strong>Status:</strong> {{ $laporan->status }}</p>
<p><strong>Tanggal Laporan:</strong>
    {{ $laporan->created_at->format('d-m-Y H:i') }}</p>

{{-- Foto bukti kondisi banjir --}}
<img src="{{ asset('storage/' . $laporan->foto) }}"
     alt="Foto laporan {{ $laporan->nama_pelapor }}"
     class="max-w-sm rounded border">

<div class="pt-4">
    <a href="{{ route('laporan.edit', $laporan) }}"
       class="text-yellow-600">Edit</a> |
    <a href="{{ route('laporan.index') }}" class="text-gray-600">Kembali</a>
</div>
```

3.10 Membuat Halaman Edit

Form edit memakai @method('PUT') karena route update memakai metode PUT. Semua input diisi dengan old() dan nilai dari $laporan. Bagian foto menampilkan foto saat ini. Kalau petugas tidak memilih file baru, kolom foto tidak diubah. Kalau petugas memilih file baru, controller menghapus file lama lalu menyimpan file baru. Status bisa diubah menjadi Menunggu, Diproses, atau Selesai.

```blade
                <div class="mb-4">
                    <x-input-label for="status" value="Status Penanganan" />
                    <select id="status" name="status"
                            class="mt-1 block w-full border-gray-300 rounded-md shadow-sm">
                        @foreach (['Menunggu', 'Diproses', 'Selesai'] as $status)
                            <option value="{{ $status }}"
                                {{ old('status', $laporan->status) === $status
                                    ? 'selected' : '' }}>
                                {{ $status }}
                            </option>
                        @endforeach
                    </select>
                    <x-input-error :messages="$errors->get('status')" class="mt-2" />
                </div>

                <div class="mb-4">
                    <p class="text-sm text-gray-600">Foto saat ini:</p>
                    <img src="{{ asset('storage/' . $laporan->foto) }}"
                         alt="Foto laporan {{ $laporan->nama_pelapor }}"
                         class="mt-2 h-32 w-32 object-cover rounded border">
                    <x-input-label for="foto"
                        value="Ganti Foto (kosongkan jika tidak ingin mengganti)"
                        class="mt-2" />
                    <input id="foto" name="foto" type="file" accept=".jpg,.jpeg,.png"
                           class="mt-1 block w-full">
                    <x-input-error :messages="$errors->get('foto')" class="mt-2" />
                </div>

                <x-primary-button>Update</x-primary-button>
                <a href="{{ route('laporan.index') }}" class="ms-3 text-gray-600">Batal</a>
            </form>

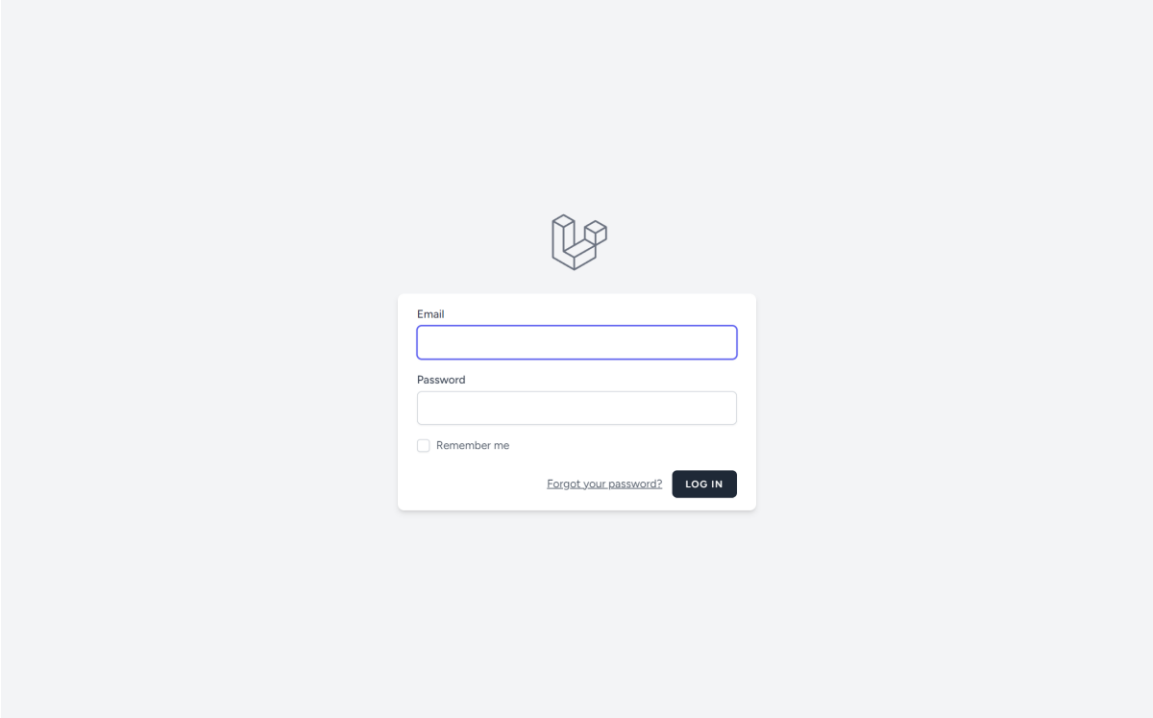
```

3.11 Menguji Aplikasi

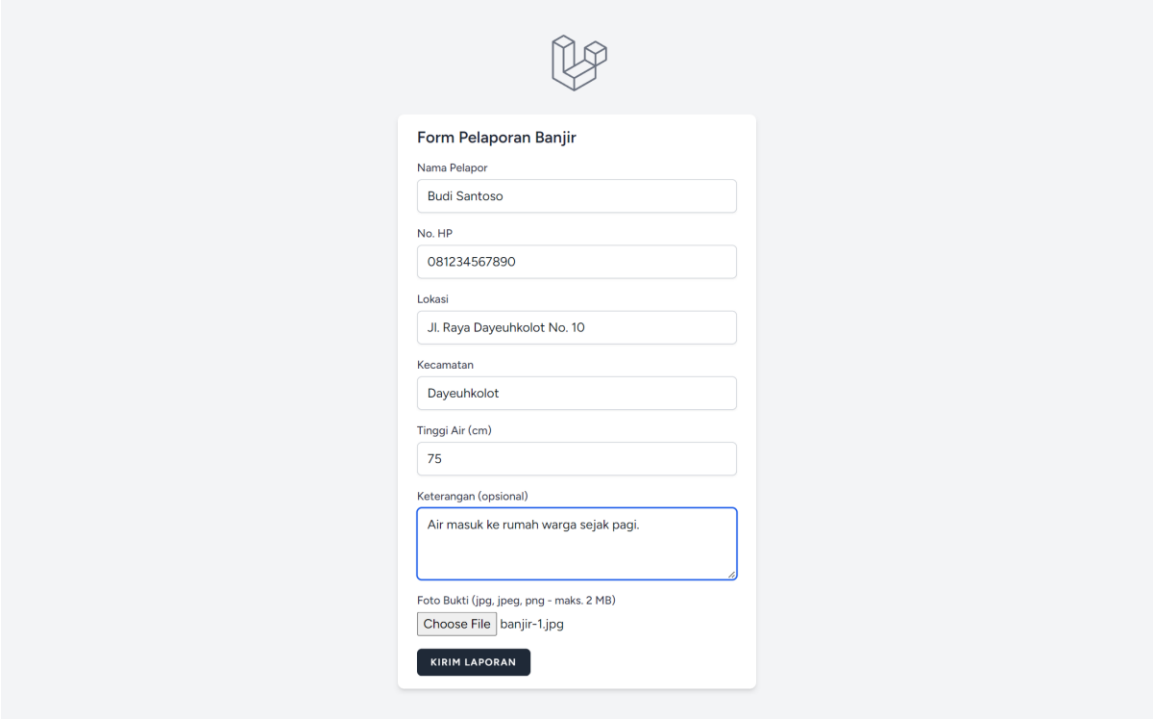
Saya menjalankan aplikasi dengan php artisan serve pada port 8000. Pengujian saya lakukan lewat browser dan phpMyAdmin. Skenario yang saya coba: mengirim laporan dengan foto jpg, mengirim file zip, mengirim foto lebih dari 2 MB, membuka /laporan tanpa login, melihat daftar dan pagination, membuka detail, mengedit status dan foto, serta menghapus laporan. Semua skenario berjalan sesuai harapan. Perintah php artisan test juga lulus 25 tes bawaan Breeze. Tes itu belum mencakup alur laporan, jadi pengujian alur utama saya lakukan secara manual.

4. Hasil dan Dokumentasi Program

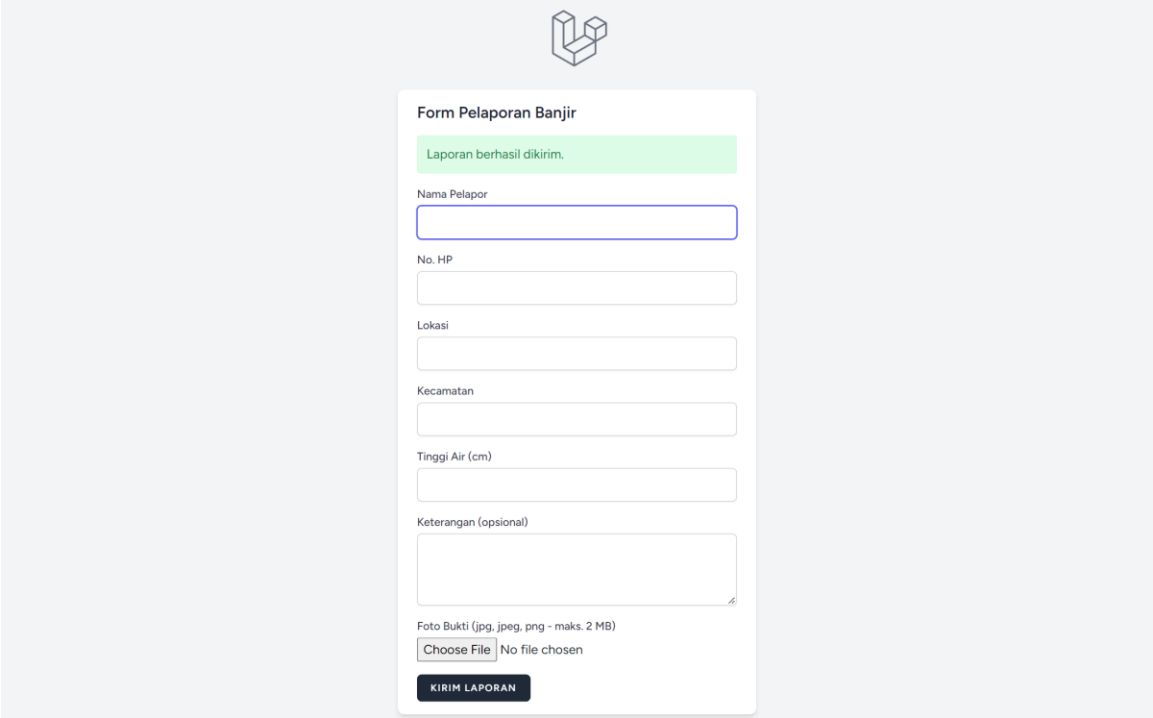
Setelah aplikasi berjalan, saya mengambil tangkapan layar dari halaman-halaman utama. Data yang tampil adalah data uji yang saya buat sendiri, bukan data warga sungguhan. Ada tujuh laporan yang saya kirim, lalu satu laporan saya hapus dan satu laporan saya ubah fotonya. Dengan begitu, jumlah laporan di akhir pengujian menjadi enam.



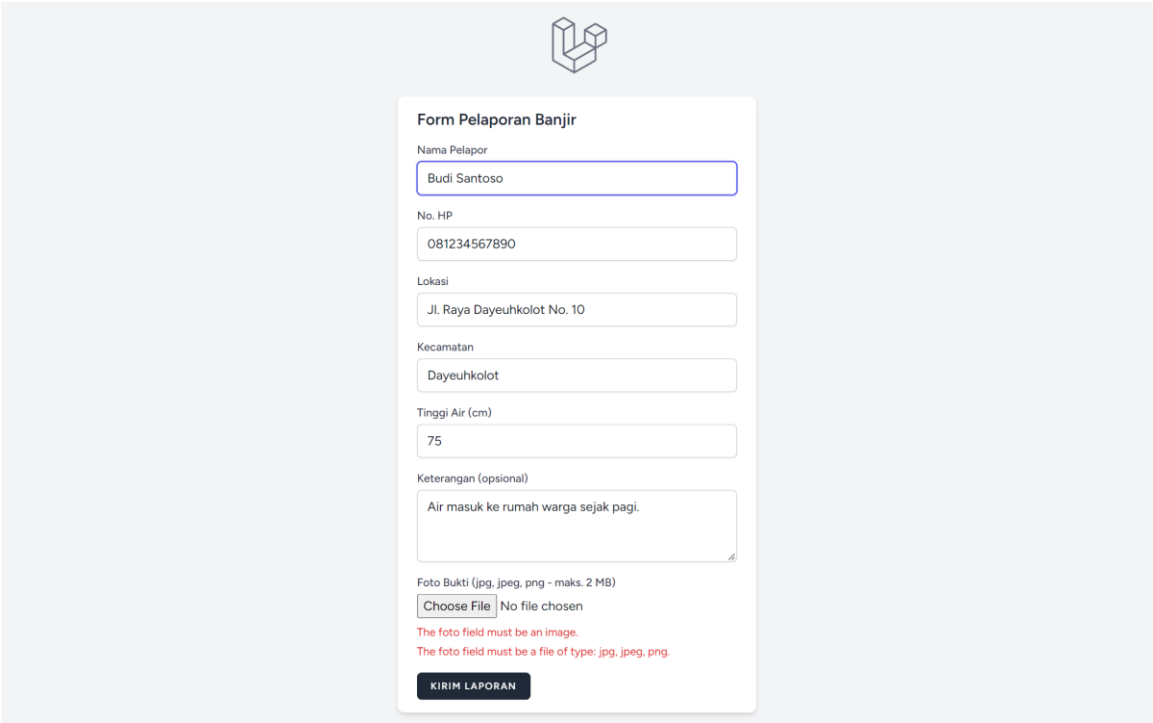
Gambar 1 menampilkan halaman login yang muncul ketika /laporan dibuka tanpa sesi. Alamat yang saya buka tetap /laporan, tetapi middleware auth mengalihkannya ke /login. Form ini berasal dari Breeze.



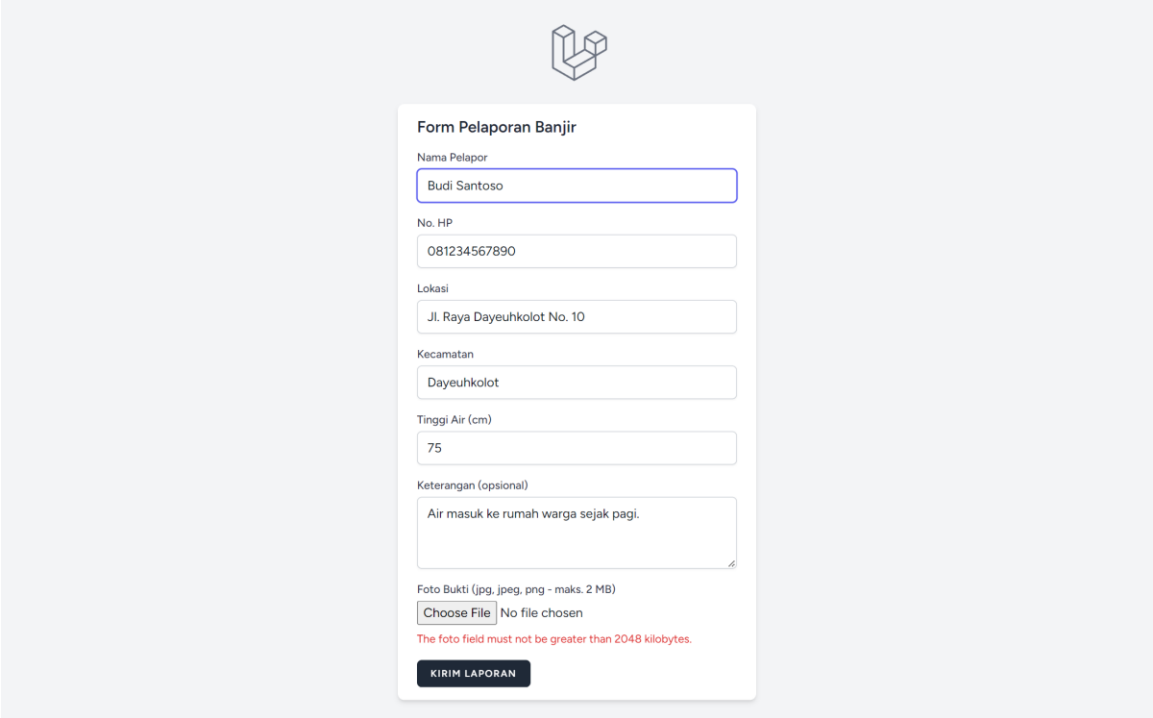
Gambar 2 menampilkan form pelaporan yang sudah diisi. Foto yang dipilih adalah gambar uji banjir-1.jpg. Form ini bisa dibuka tanpa login.



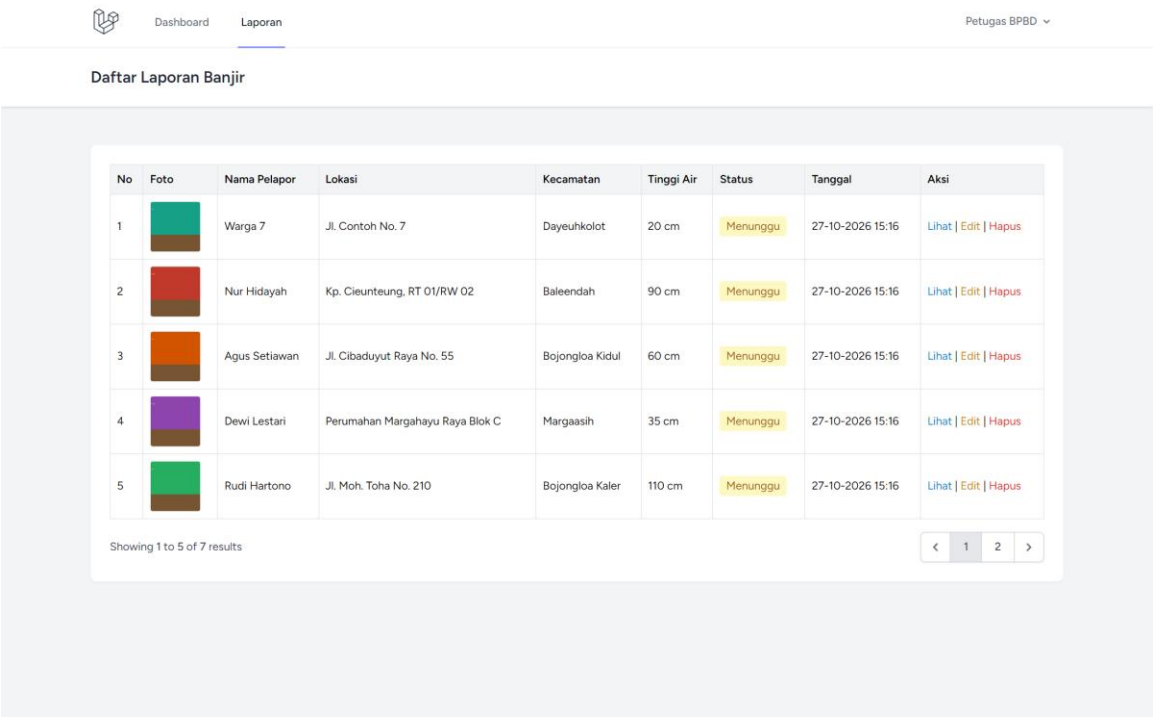
Gambar 3 menampilkan pesan 'Laporan berhasil dikirim.' setelah form dikirim. Data langsung masuk ke tabel laporan dan file foto tersimpan di storage.



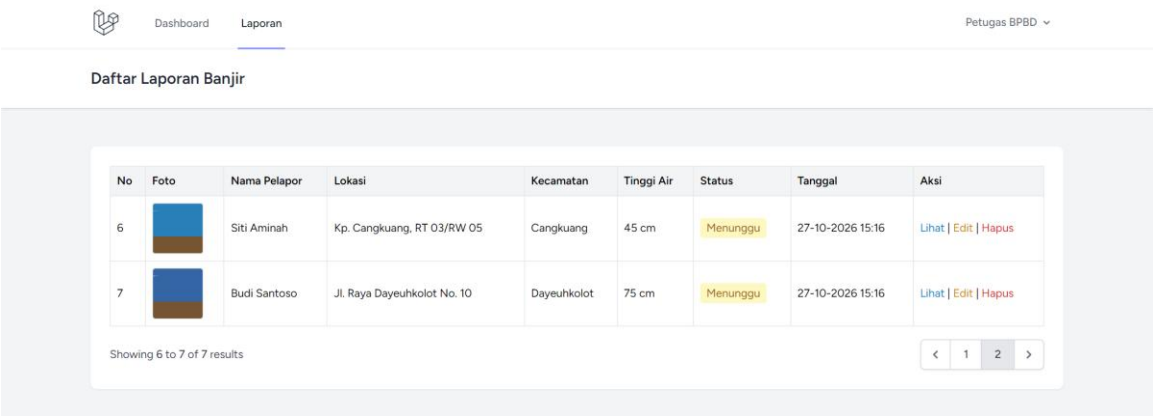
Gambar 4 menampilkan pesan galat ketika saya mengunggah file zip. Laravel menolak file itu karena bukan gambar dan jenisnya tidak termasuk jpg, jpeg, atau png.



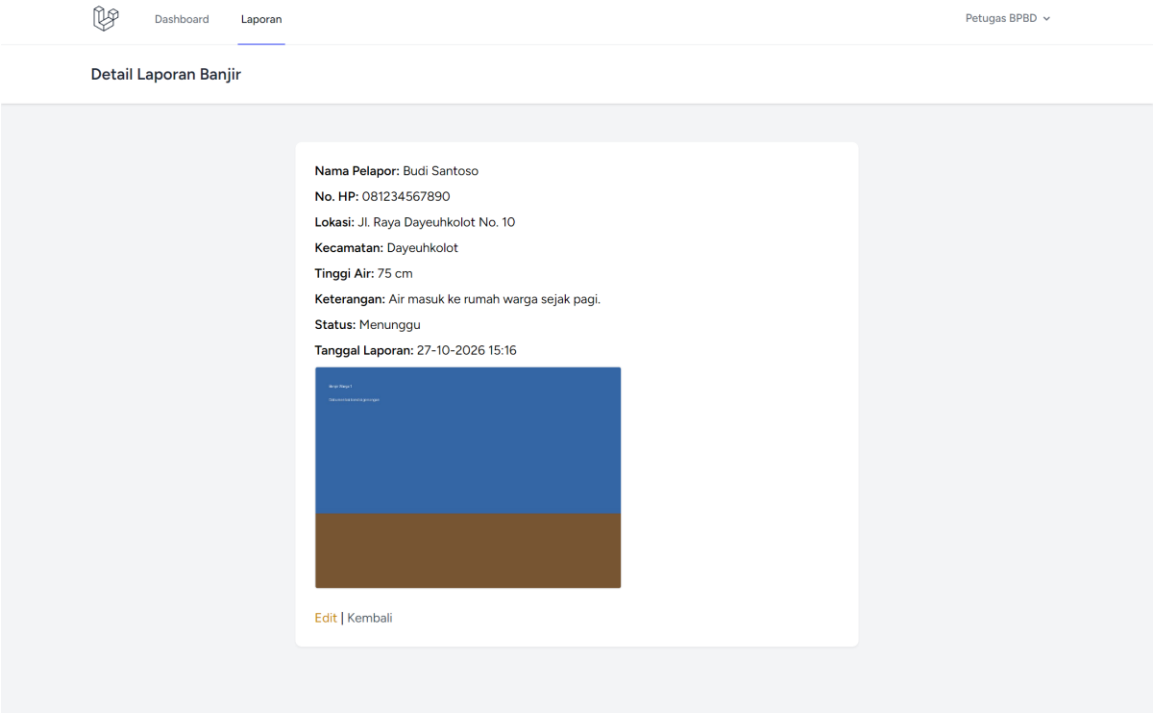
Gambar 5 menampilkan pesan galat ketika foto yang saya unggah berukuran lebih dari 2 MB. Pesan muncul di bawah input foto, tepat di atas tombol Kirim Laporan.



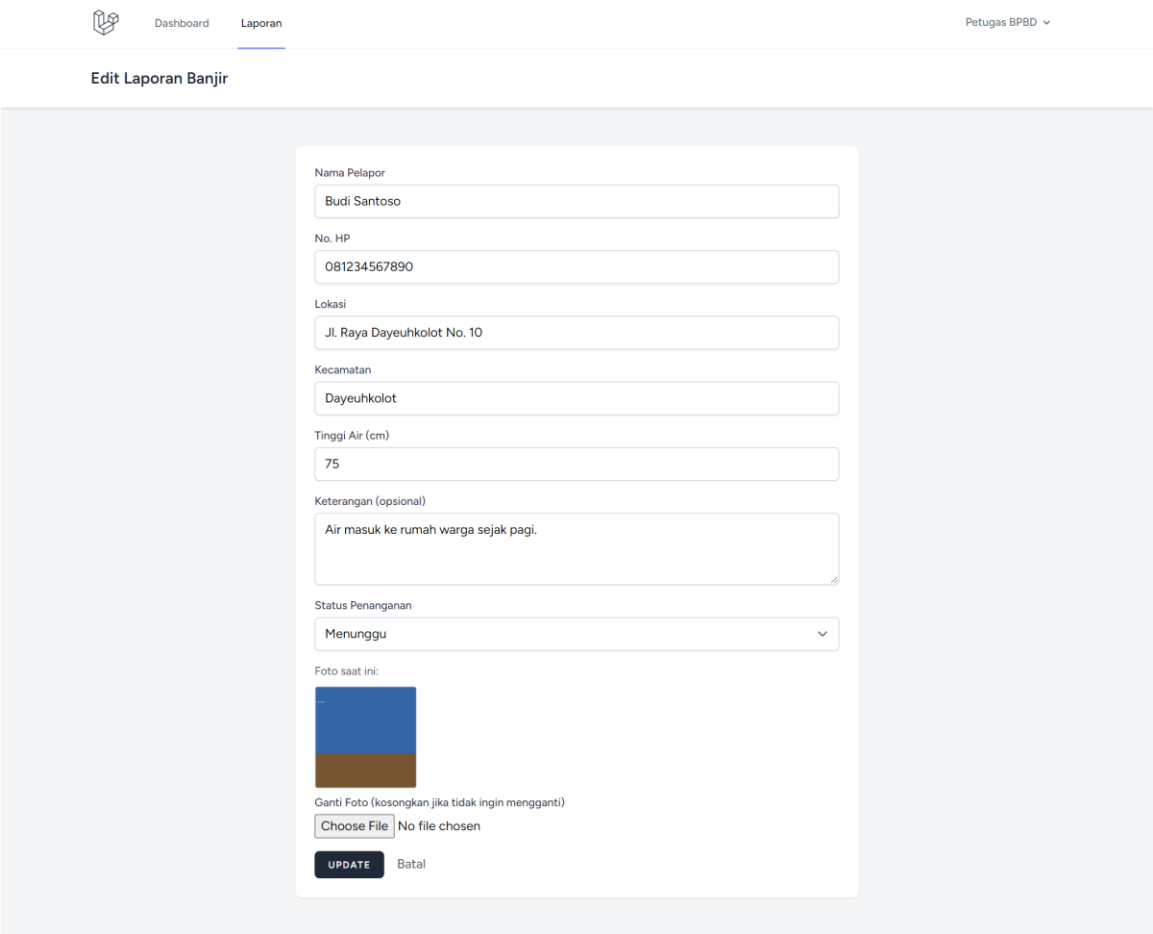
Gambar 6 menampilkan daftar laporan halaman pertama. Ada lima baris dengan thumbnail, status berwarna, dan tombol aksi. Di bawah tabel terlihat tautan pagination ke halaman 2.



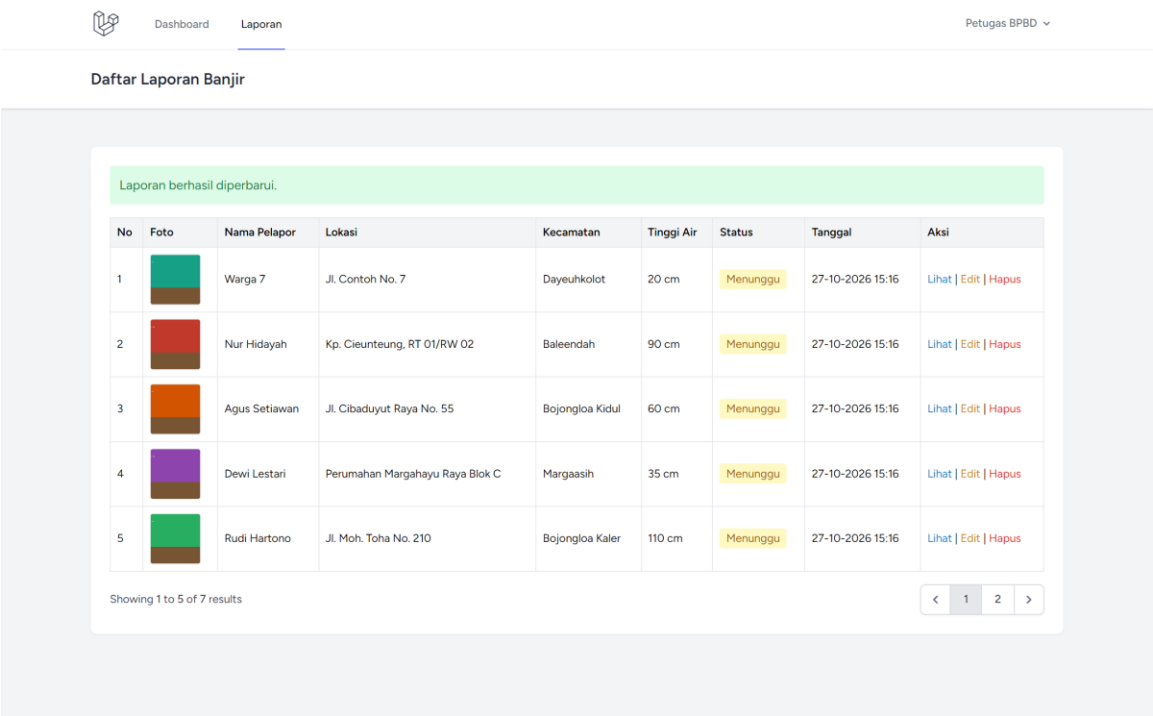
Gambar 7 menampilkan halaman kedua. Laporan yang tersisa tetap tampil dengan format yang sama. Pagination membagi tujuh laporan menjadi dua halaman karena index memakai paginate(5).



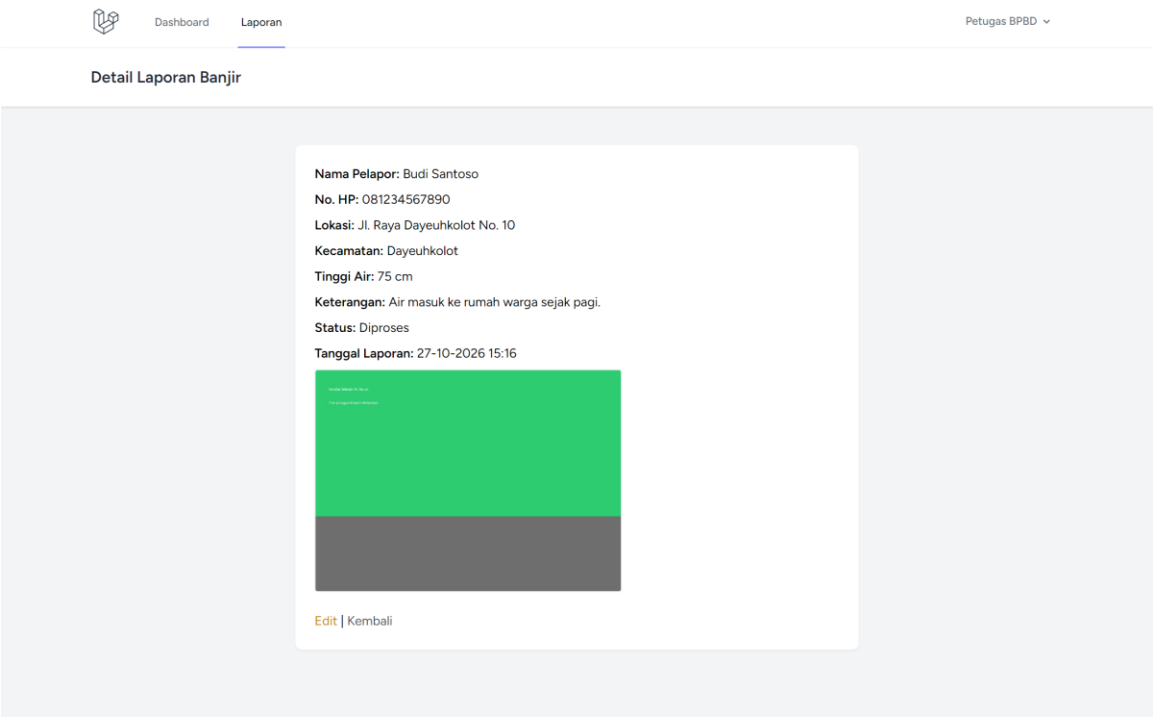
Gambar 8 menampilkan detail laporan milik Budi Santoso. Semua kolom tampil, termasuk foto bukti berukuran penuh di dalam batas max-w-sm.



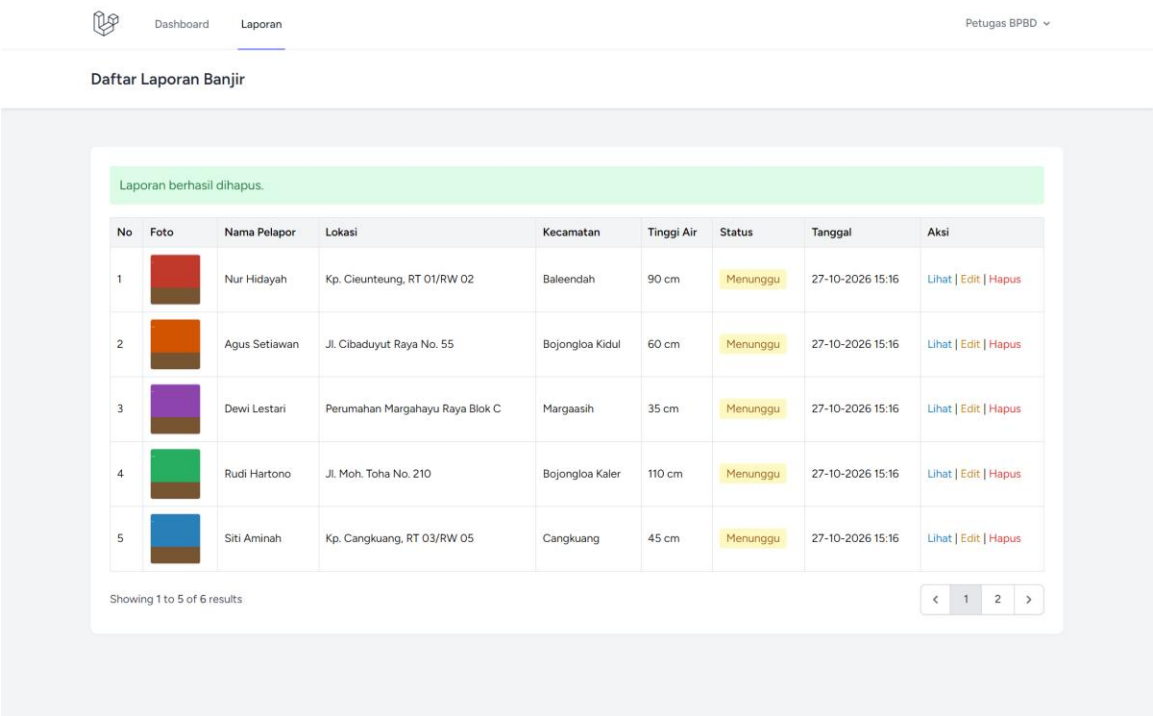
Gambar 9 menampilkan form edit sebelum foto diganti. Status masih Menunggu dan foto saat ini adalah foto pertama. Di bawah foto ada input untuk memilih file baru.



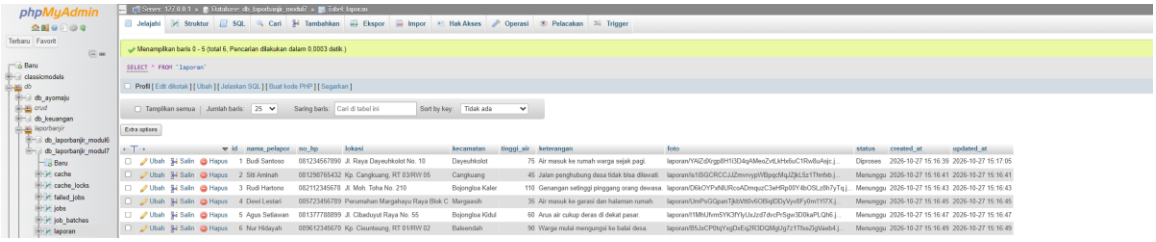
Gambar 10 menampilkan pesan 'Laporan berhasil diperbarui.' setelah saya mengubah status menjadi Diproses dan memilih foto baru. Pesan tampil di atas tabel.



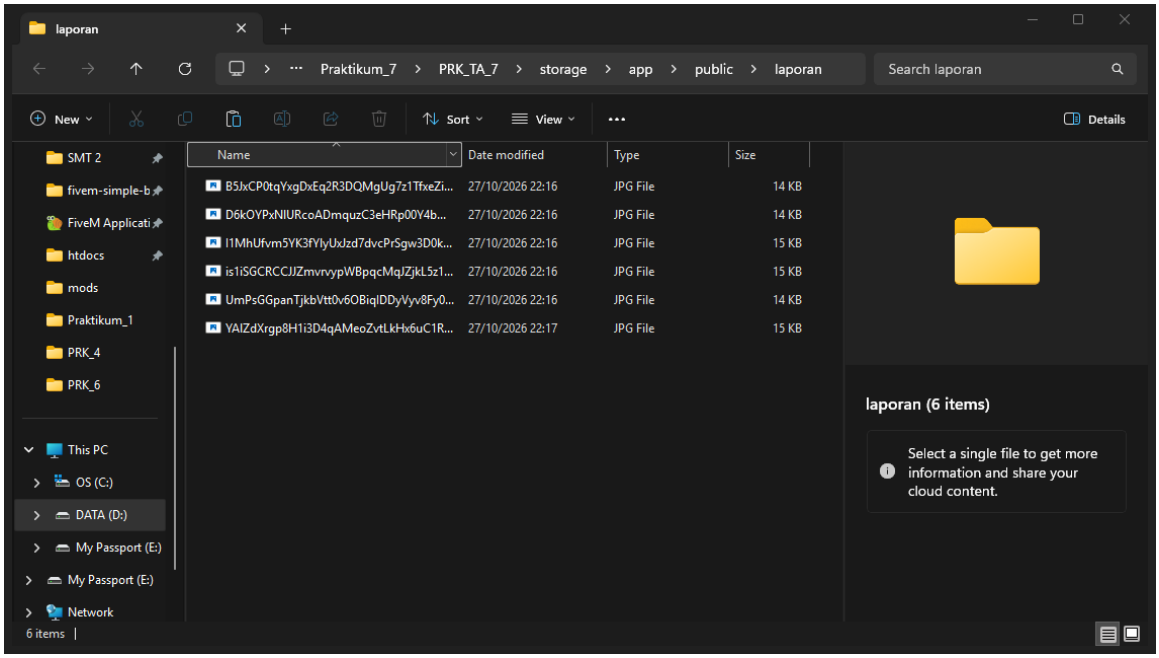
Gambar 11 menampilkan detail laporan setelah perubahan. Status sudah menjadi Diproses dan foto yang tampil adalah foto pengganti. Foto lama sudah tidak ada di folder storage.



Gambar 12 menampilkan pesan 'Laporan berhasil dihapus.' dan daftar yang sekarang berisi enam laporan. Laporan contoh milik Warga 7 sudah hilang dari tabel.



Gambar 13 menampilkan isi tabel laporan di phpMyAdmin. Ada enam baris dengan kolom foto yang berisi path file, misalnya laporan/YAlZ....jpg. Kolom status menunjukkan satu laporan Diproses dan lima laporan Menunggu.



Gambar 14 menampilkan isi folder storage/app/public/laporan. Ada enam file dengan nama acak. Jumlah file sama dengan jumlah baris di tabel karena satu foto lama sudah dihapus saat penggantian dan satu foto ikut terhapus saat laporan dihapus.

5. Analisis dan Pembahasan

Bagian ini membahas hal-hal yang saya temui selama mengerjakan tugas akhir, mulai dari alur CRUD sampai kendala teknis.

1. Pemetaan CRUD ke route dan method controller.

Route::resource memetakan tujuh route ke tujuh method. Di aplikasi ini resource dipisah menjadi dua bagian. only(['create','store']) dipakai untuk form warga, sedangkan except(['create','store']) dipakai untuk pengelolaan petugas. Pemisahan itu membuat satu resource tetap bisa diakses dari dua kelompok pengguna tanpa menulis route manual satu per satu.

2. Penyimpanan foto dan path di database.

Foto disimpan dengan store('laporan','public'). Method itu mengembalikan path relatif seperti laporan/xxxx.jpg. Yang saya simpan di kolom foto adalah path tersebut, bukan nama file saja. Di view, path itu digabung dengan asset('storage/...') sehingga URL-nya menjadi /storage/laporan/xxxx.jpg. Pola ini membuat file bisa dipindah atau diganti tanpa mengubah struktur tabel. Saat foto diganti, saya menghapus file lama dengan Storage::disk('public')->delete($laporan->foto) sebelum menyimpan file baru. Saat laporan dihapus, file fisik juga dihapus lebih dulu. Folder storage pada Gambar 14 berisi enam file, sama dengan enam laporan yang tersisa.

3. Validasi dan pembatasan akses.

Validasi foto memakai image, mimes:jpg,jpeg,png, dan max:2048. Aturan image memeriksa isi file, sedangkan mimes memeriksa ekstensi dan tipe yang diizinkan. Pesan galat ditampilkan di bawah masing-masing input. Untuk akses, route pengelolaan berada di middleware auth. Warga tidak perlu akun, tetapi juga tidak bisa membuka daftar, detail, atau form edit. Pembatasan ini saya uji dengan membuka /laporan di jendela tanpa sesi. Hasilnya muncul halaman login seperti pada Gambar 1.

4. Pagination dan tampilan daftar.

index memakai latest()->paginate(5). Dengan tujuh laporan, halaman pertama berisi lima baris dan halaman kedua berisi dua baris. Thumbnail memakai object-cover agar rasio foto tetap rapi. Status diberi warna yang berbeda supaya petugas bisa melihat prioritas dengan cepat. Tombol aksi diletakkan di kolom terakhir agar tabel tidak terlalu lebar.

5. Kendala dan penyelesaian.

Ada dua kendala yang benar-benar saya temui. Pertama, server MySQL berhenti saat saya menyiapkan tangkapan layar. Laravel menampilkan SQLSTATE[HY000] [2002] Connection refused karena tidak bisa terhubung ke 127.0.0.1:3306. Saya menyalakan ulang mysqld dari XAMPP, lalu aplikasi kembali normal. Kedua, beberapa kelas Tailwind dari view baru belum masuk ke CSS hasil build. Akibatnya foto di halaman detail tampil terlalu besar dan badge status tidak berwarna. Saya menjalankan npm run build lagi setelah view selesai dibuat, lalu tampilan kembali sesuai. Kejadian ini menunjukkan bahwa build aset perlu diulang setiap kali ada kelas Tailwind baru di Blade.

6. Alasan memakai Laravel Breeze.

Saya memilih Breeze karena aplikasi ini membutuhkan autentikasi petugas, tetapi tidak membutuhkan pengelolaan peran yang rumit. Breeze sudah menyediakan halaman login, register, profil, layout Blade, komponen input, dan tes autentikasi. Dengan begitu, waktu saya lebih banyak dipakai untuk alur laporan. Breeze juga memakai Tailwind, jadi tampilan form warga dan halaman petugas bisa memakai komponen yang sama. Kalau autentikasi ditulis dari nol, saya harus membuat controller, route, view, dan validasi login sendiri. Untuk kebutuhan tugas akhir ini, Breeze sudah cukup.

7. Perbedaan dengan latihan modul.

Latihan modul memakai satu tabel uploads yang terhubung ke user_id dan seluruh CRUD berada di dalam middleware auth. Di tugas akhir ini, laporan warga tidak terikat akun karena warga mengirim tanpa login. Karena itu saya tidak memakai user_id, tetapi menambahkan kolom status untuk penanganan petugas. Foto wajib di store, tetapi opsional di update. Route resource juga dipisah menjadi dua kelompok. Perbedaan ini saya ambil dari teks tugas, bukan dari modul. Nama database juga saya ubah menjadi db_laporbanjir_modul7 agar tidak bentrok dengan database modul sebelumnya.

6. Kesimpulan

Aplikasi LaporBanjir berhasil dibuat dengan Laravel 12 dan Laravel Breeze. Warga bisa mengirim laporan dari form publik, sedangkan petugas mengelola laporan setelah login.

1. Tabel laporan, model Laporan, dan LaporanController dengan tujuh method resource sudah berjalan.
2. Foto tersimpan di storage/app/public/laporan, path-nya tercatat di database, dan public/storage sudah tertaut.
3. Validasi jpg, jpeg, png, dan batas 2 MB berfungsi. File zip dan foto lebih dari 2 MB ditolak dengan pesan di bawah input.
4. Halaman daftar menampilkan thumbnail, status, pagination, dan tombol aksi. Halaman detail dan edit menampilkan data serta foto dengan benar.
5. Penggantian foto menghapus file lama, dan penghapusan laporan menghapus file fisiknya.
6. Route pengelolaan terlindungi middleware auth, sedangkan form warga tetap terbuka.
7. Laravel Breeze mempercepat pembuatan autentikasi dan menyediakan layout yang konsisten.

Pengujian manual mencakup kirim laporan, validasi, pagination, detail, edit, ganti foto, dan hapus. Perintah php artisan test lulus 25 tes bawaan Breeze. Tes khusus untuk alur laporan belum saya tulis, jadi pengujian alur utama masih mengandalkan langkah manual yang saya dokumentasikan di bagian 4.

7. Link GitHub

https://github.com/davinaileenskie1-hue/PABW-Semester-3